TherMOS – Thermal Model Order Reduction

Theory guide to the model order reduction options of the toolkit

Dr. Pascal A. Schirmer

Version 1.0, 9 November 2026

Abstract

A detailed thermal model of a power electronic component – a finite element (FE) or finite volume (FV) model with thousands of nodes – is accurate but far too slow for mission-profile simulations, lifetime estimation, design optimisation or a temperature observer in a controller. Model order reduction (MOR) replaces it by a model with a few states that reproduces the temperatures of interest. TherMOS offers eight reduced model families in its operating modes (RC, SS, SF, PO, PS, ML, DL, PI) and two more in the board designer of the graphical user interface. They differ in what they need (material data, measured profiles, single-source experiments), what they return (one temperature, several junctions, the full field) and how safely they extrapolate. This guide derives every option from one common full-order model, states every equation with its symbols and units, illustrates the key ideas with worked examples computed with the toolkit, and ends with a selection guide.

Contents

1 How to read this guide **3**

2 The full-order model **3**

2.1 Heat equation 3

2.2 Semi-discrete form 3

2.3 Modal form: why thermal models can be reduced 4

3 Thermal networks: Foster and Cauer (RC mode) **5**

3.1 Foster network 5

3.2 Fit of the Foster parameters 6

3.3 Cauer network and structure function 6

4 State-space identification (SS mode) **7**

4.1 Model 7

4.2 Subspace identification (N4SID/MOESP) 7

4.3 Output-error refinement 8

5 Coupled heat sources (SF mode) **8**

5.1 Method 2 (default): coupled Cauer network fitted to any profile 9

5.2 Method 1 (MATLAB): nodal Foster model 9

6 Proper orthogonal decomposition and Galerkin projection (PO mode) 10

6.1 Proper orthogonal decomposition (POD) 10

6.2 Galerkin projection 10

6.3 Calibration protocols 11

7 POD state space (PS mode) 12

8 Reduced models of the board designer 13

8.1 POD-Galerkin on the FV grid 13

8.2 Coupled Cauer network from step responses 13

8.3 Electro-thermal coupling 13

9 Data-driven models (ML and DL modes) 13

9.1 Static regression with smoothed features (ML mode) 13

9.2 Sequence networks (DL mode) 14

10 Physics-informed neural network (PI mode) 14

11 Comparison and selection guide 15

12 Error metrics 16

1 How to read this guide

Section 2 defines the full-order model that every reduced model approximates and shows its modal form. This form is the key to the whole guide: it explains why a thermal system can be reduced at all and why Foster networks, state-space models and POD models are three views of the same thing. Sections 3 to 5 treat the *lumped* models that reproduce a few temperatures (RC, SS, SF modes). Sections 6 to 8 treat the *field* models that reproduce the whole temperature field (PO and PS modes, board designer). Sections 9 and 10 treat the *data-driven* and *hybrid* models (ML, DL, PI modes). Section 11 compares all options and gives a selection guide; Section 12 defines the error metrics. Every section ends with a box that names the configuration parameters (sheet `Mdl` of the configuration workbook, GUI view *Parameters*).

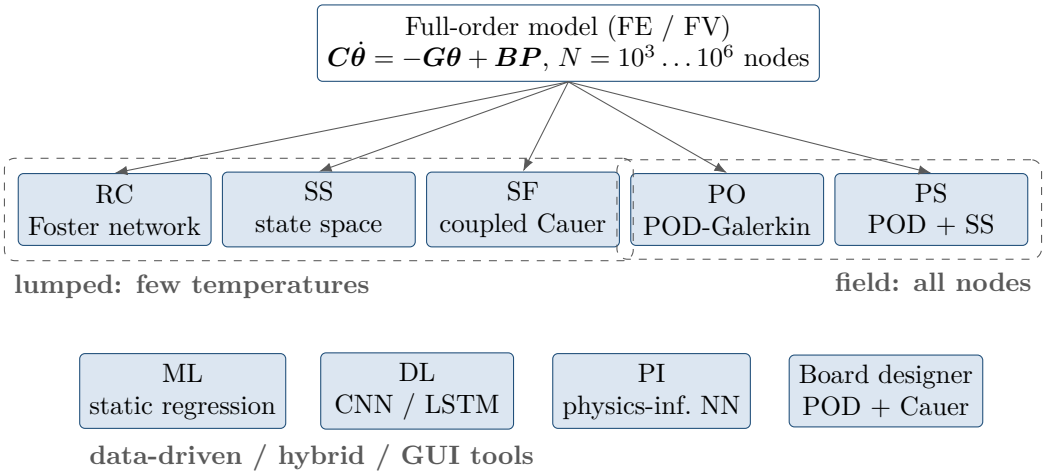


Figure 1: Reduced model families of TherMOS. The lumped and field models approximate the full-order model directly (by fitting its responses or by projecting its equations). The data-driven models learn the input–output map from profiles; the PINN adds the physics of a lumped network to the training. The board designer builds its own FV model and reduces it with POD and a coupled Cauer network.

2 The full-order model

2.1 Heat equation

The temperature $T(\boldsymbol{x}, t)$ in a solid with volumetric heat source Q follows the heat equation

$$\rho c \, \frac{\partial T}{\partial t} = \nabla \cdot (k \, \nabla T) - h_v \, (T - T_a) + Q(\boldsymbol{x}, t), \tag{1}$$

with convection $-k \, \partial T / \partial n = h \, (T - T_{\mathrm{ref}})$ or a fixed temperature on parts of the boundary and an adiabatic rest. The distributed sink h_v is used for thin boards whose cooling through the bottom side is smeared over the area (board designer, Section 8).

2.2 Semi-discrete form

An FE or FV discretisation of (1) gives a linear system of N ordinary differential equations

$$\boldsymbol{C} \, \dot{\boldsymbol{\theta}}(t) = -\boldsymbol{G} \, \boldsymbol{\theta}(t) + \boldsymbol{B} \, \boldsymbol{P}(t), \qquad \boldsymbol{y}(t) = \boldsymbol{L} \, \boldsymbol{\theta}(t). \tag{2}$$

Table 1: Symbols of the full-order model.

Symbol	Unit	Meaning (typical value)
$T,\ T_a,\ T_{\mathrm{ref}}$	°C	temperature; ambient; reference temperature (coolant, heat sink, ambient)
$\theta = T - T_{\mathrm{ref}}$	K	temperature rise over the reference; all linear models work with θ
ρc	$\mathrm{J/(m^3\,K)}$	volumetric heat capacity (Cu 3.4×10^6, Si 1.6×10^6, FR4 1.9×10^6)
k	$\mathrm{W/(m\,K)}$	thermal conductivity (Cu 390, Si 150, Al_2O_3 25, FR4 0.3 through-plane)
$h,\ h_v$	$\mathrm{W/(m^2\,K)}$, $\mathrm{W/(m^3\,K)}$	surface heat transfer coefficient (air 5–50, liquid cold plate 1000–20000); distributed sink per volume
Q	$\mathrm{W/m^3}$	volumetric heat generation (losses of a chip divided by its volume)
N	–	number of nodes of the discretised model (10^3–10^6)
S	–	number of loss sources (chips, windings)
$\boldsymbol{\theta} \in \mathbb{R}^N$	K	node temperature rises
$\boldsymbol{P} \in \mathbb{R}^S$	W	losses of the sources
$\boldsymbol{C} \in \mathbb{R}^{N\times N}$	J/K	capacitance matrix: diagonal (FV) or consistent mass matrix (FE), positive definite
$\boldsymbol{G} \in \mathbb{R}^{N\times N}$	W/K	conductance matrix: symmetric, positive definite if any heat leaves the body
$\boldsymbol{B} \in \mathbb{R}^{N\times S}$	–	input matrix: share of loss j that enters node i (columns sum to 1)
$\boldsymbol{L} \in \mathbb{R}^{p\times N}$	–	output matrix: selects (or averages) the p temperatures of interest

For the FV model of the board designer (`calcBoardMat`) every node i owns the control volume V_i; the entries are

$$C_i = (\rho c)_i V_i, \qquad G_{ij} = -\frac{A_{ij}}{d/(2k_i) + d/(2k_j)}\ (i \neq j \text{ neighbours}), \qquad G_{ii} = \sum_{j\neq i} |G_{ij}| + h_{v,i} V_i, \quad (3)$$

with A_{ij} the face between the volumes and d the node distance. Three properties hold for every thermal model and are used throughout the guide:

1. **Passivity.** $\boldsymbol{C}$ and $\boldsymbol{G}$ are symmetric positive definite. The stored energy $\frac{1}{2}\boldsymbol{\theta}^{\mathsf{T}}\boldsymbol{C}\boldsymbol{\theta}$ can only fall without losses: the model is stable and does not oscillate.
2. **Linearity.** With temperature-independent ρc, k, h the response to several sources is the sum of the responses to each source (superposition). Thermal impedances and resistance matrices rely on it.
3. **Steady state.** For constant losses $\boldsymbol{\theta}_\infty = \boldsymbol{G}^{-1}\boldsymbol{B}\boldsymbol{P}$. The thermal resistance matrix $R_{\mathrm{th}} = \boldsymbol{L}\boldsymbol{G}^{-1}\boldsymbol{B}$ (K/W) collects self-heating (diagonal) and cross-heating (off-diagonal).

2.3 Modal form: why thermal models can be reduced

The generalised eigenvalue problem

$$\boldsymbol{G}\,\boldsymbol{v}_k = \lambda_k\,\boldsymbol{C}\,\boldsymbol{v}_k, \qquad \boldsymbol{V}^{\mathsf{T}}\boldsymbol{C}\boldsymbol{V} = \boldsymbol{I}, \quad \boldsymbol{V}^{\mathsf{T}}\boldsymbol{G}\boldsymbol{V} = \boldsymbol{\Lambda} = \mathrm{diag}(\lambda_1, \ldots, \lambda_N), \quad 0 < \lambda_1 \leq \lambda_2 \leq \ldots \quad (4)$$

has N real positive eigenvalues λ_k (1/s) because $\boldsymbol{C}$ and $\boldsymbol{G}$ are symmetric positive definite. With $\boldsymbol{\theta} = \boldsymbol{V}\boldsymbol{z}$, (2) falls apart into N independent first-order systems $\dot{z}_k = -\lambda_k z_k + \boldsymbol{v}_k^{\mathsf{T}}\boldsymbol{B}\boldsymbol{P}$. The response of output i to a 1 W step in source j – the *thermal impedance* – is therefore a sum of exponentials:

$$\boxed{Z_{ij}(t) = \sum_{k=1}^{N} R_{ijk}\left(1 - e^{-t/\tau_k}\right), \qquad \tau_k = \frac{1}{\lambda_k}, \qquad R_{ijk} = \frac{(\boldsymbol{L}\boldsymbol{v}_k)_i\,(\boldsymbol{v}_k^{\mathsf{T}}\boldsymbol{B})_j}{\lambda_k}} \quad (5)$$

This is a Foster network with N RC pairs. Its time constants τ_k are the same for all inputs and outputs; only the weights R_{ijk} change. For self-heating with collocated input and output ($\boldsymbol{L} = \boldsymbol{B}^{\mathsf{T}}$) every $R_{iik} = (\boldsymbol{v}_k^{\mathsf{T}}\boldsymbol{b}_i)^2/\lambda_k$ is positive, for cross-heating ($i \neq j$) single terms can be negative.

Why a few states are enough

Two facts make the reduction work. (1) The time constants are spread over many decades (chip 1 ms to 10 ms, substrate 0.1 s to 1 s, heat sink 10 s to 1000 s); a sampled model with $T_s = 1\,\mathrm{s}$ cannot see modes with $\tau_k \ll T_s$ – they are quasi-static. (2) Most modes are hardly excited by the chip losses or hardly seen at the sensors: their weights R_{ijk} are tiny. Typically 3–8 modes carry more than 99.9 % of the response. Every reduced model in this guide is a way to find these few modes – by fitting (RC, SS, SF, PS), by projection (PO) or by learning (ML, DL, PI).

Worked example 1: two-node chain

A chip ($C_1 = 2\,\mathrm{J/K}$) sits on a heat sink ($C_2 = 40\,\mathrm{J/K}$) with $R_{12} = 0.5\,\mathrm{K/W}$ between them and $R_{2a} = 1.0\,\mathrm{K/W}$ to the coolant:

$$\boldsymbol{C} = \begin{bmatrix} 2 & 0 \\ 0 & 40 \end{bmatrix}, \qquad \boldsymbol{G} = \begin{bmatrix} 2 & -2 \\ -2 & 3 \end{bmatrix} \mathrm{W/K}, \qquad \boldsymbol{B} = \boldsymbol{L}^{\mathsf{T}} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}.$$

(4) gives $\lambda_1 = 0.0238/\mathrm{s}$, $\lambda_2 = 1.051/\mathrm{s}$, i.e. $\tau_1 = 42.0\,\mathrm{s}$ (heat sink) and $\tau_2 = 0.951\,\mathrm{s}$ (chip). The Foster weights of the chip are $R_1 = 1.048\,\mathrm{K/W}$ and $R_2 = 0.452\,\mathrm{K/W}$; their sum is the steady-state resistance $[\boldsymbol{G}^{-1}]_{11} = R_{12} + R_{2a} = 1.5\,\mathrm{K/W}$. Note that neither Foster pair equals a physical part: $\tau_2 = 0.951\,\mathrm{s}$ is not $R_{12}C_1 = 1\,\mathrm{s}$, and $R_1 \neq R_{2a}$. Figure 2 shows the curve.

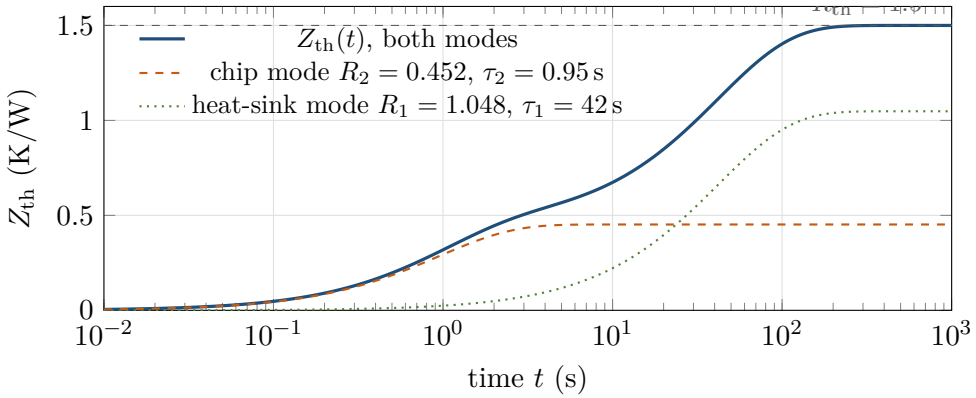


Figure 2: Thermal impedance of worked example 1 on a logarithmic time axis. Each Foster term is an S-curve centred on its time constant; the chip mode settles after about $3\tau_2 \approx 3\,\mathrm{s}$, the heat-sink mode after about $3\tau_1 \approx 2\,\mathrm{min}$. After about 200 s the curve reaches $R_{\mathrm{th}} = 1.5\,\mathrm{K/W}$: 10 W of losses heat the chip by 15 K.

3 Thermal networks: Foster and Cauer (RC mode)

3.1 Foster network

The RC mode reproduces one temperature (the first output channel) from one loss with a Foster network of K RC pairs in series, i.e. (5) truncated to the K dominant modes:

$$Z_{\mathrm{th}}(t) = \sum_{k=1}^{K} R_k \left(1 - e^{-t/\tau_k}\right), \qquad \tau_k = R_k C_k, \qquad R_{\mathrm{th}} = \sum_k R_k. \tag{6}$$

For an arbitrary loss profile the response is the convolution of the impulse response $\mathrm{d}Z_{\mathrm{th}}/\mathrm{d}t$ with $P(t)$. TherMOS evaluates it recursively for every pair (`rcSim`, trapezoidal/Tustin rule, exact for

losses that are linear between the samples):

$$\theta_k(n) = \frac{2\tau_k - T_s}{2\tau_k + T_s}\,\theta_k(n-1) + \frac{R_k T_s}{2\tau_k + T_s}\big(P(n) + P(n-1)\big), \qquad T(n) = T_{\mathrm{ref}}(n) + \sum_{k=1}^{K} \theta_k(n). \quad (7)$$

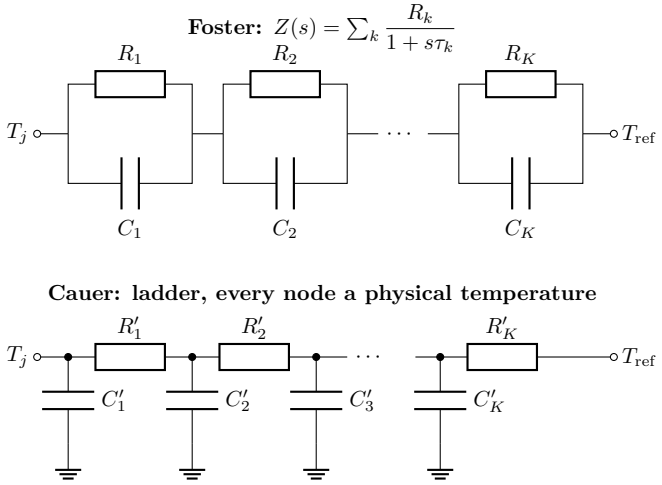


Figure 3: Foster (top) and Cauer (bottom) form of a heat path from the junction T_j to the reference T_{ref}. Both have the same impedance $Z(s)$ when the Cauer parameters are computed from the Foster ones. The capacitances of the Cauer ladder are grounded (connected to thermal ground, 0 K rise), those of the Foster network sit across the resistances – they have no physical meaning.

3.2 Fit of the Foster parameters

TherMOS fits the $2K$ parameters by an *output-error* least-squares fit: the network (7) is simulated with the measured losses of all training profiles and compared with the measured temperature rise $\Delta T(n) = T(n) - T(0)$:

$$\min_{\ln R_k,\ \ln \tau_k} \sum_{\text{profiles}} \sum_{n} \big(\theta_{\mathrm{sim}}(n) - \Delta T_{\mathrm{meas}}(n)\big)^2. \quad (8)$$

Logarithmic parameters keep R_k and τ_k positive and give the time constants that span several decades comparable step sizes; the start values of τ_k are spread logarithmically between T_s and the profile length. Unlike the classical fit of (6) to $\Delta T/P$ (MATLAB version, `legacy = 1`) the output-error fit works for any loss profile, not only for a constant step.

What the fit can and cannot identify. R_{th} is always well determined. The split into pairs is not unique: pairs with τ_k within a factor of about 3 are hard to separate, and time constants that the profile does not excite (shorter than T_s, longer than the profile) cannot be found. Pulse profiles excite the fast modes better than a single step.

3.3 Cauer network and structure function

A Cauer ladder (Fig. 3, bottom) has the same impedance as a Foster network when its parameters follow from the continued-fraction expansion of the Foster admittance (`foster2cauer`):

$$Z(s) = \sum_{k=1}^{K} \frac{R_k}{1 + s\tau_k} = \cfrac{1}{sC'_1 + \cfrac{1}{R'_1 + \cfrac{1}{sC'_2 + \cfrac{1}{R'_2 + \cdots}}}} \quad (9)$$

For worked example 1 the expansion returns exactly the physical ladder $R' = (0.5,\, 1.0)\,\mathrm{K/W}$, $C' = (2,\, 40)\,\mathrm{J/K}$: a Cauer ladder is unique and its nodes can be physical temperatures if the heat path really is one-dimensional. Plotting the cumulative capacitance $\sum_{k\le m} C'_k$ over the cumulative resistance $\sum_{k\le m} R'_k$ gives the *cumulative structure function* (JEDEC JESD51-14 [6]); flat parts are layers with a high resistance and little mass (solder, thermal interface), steep parts are heavy layers with a low resistance (baseplate, heat sink). The GUI page *Theory & Tools › Thermal networks* converts Foster to Cauer and draws the structure function.

Interpretation

Use a Foster network to *simulate* a heat path (simple, exact for the fitted path). Convert it to Cauer to *connect* paths (e.g. a module network to a heat-sink network) or to *interpret* it (structure function): Foster networks cannot be connected in series because their inner nodes are not temperatures.

Configuration in TherMOS

`Mdl › rc › K`: number of RC pairs K (3–5 for chip, solder, substrate, heat sink). `Mdl › gen › opt = 1`: order search $K = 1 \ldots$ `Kmax` on the validation data. `Mdl › rc › bc`: parameter bound of the MATLAB fit (`legacy = 1` only). Only the first input/output pair is used; for several coupled outputs use the SS or SF mode.

4 State-space identification (SS mode)

4.1 Model

The SS mode identifies one multiple-input multiple-output (MIMO) model for all m losses and p temperatures:

$$\boldsymbol{x}(n+1) = \boldsymbol{A}\,\boldsymbol{x}(n) + \boldsymbol{B}_d\,\boldsymbol{u}(n), \qquad \boldsymbol{y}(n) = \boldsymbol{C}_o\,\boldsymbol{x}(n), \tag{10}$$

with $\boldsymbol{u} = \boldsymbol{P}$ (W), $\boldsymbol{y} = \boldsymbol{\theta}$ (K), n_x states $\boldsymbol{x}$ without physical meaning and no direct feedthrough. A continuous model $\dot{\boldsymbol{x}} = \boldsymbol{A}_c\boldsymbol{x} + \boldsymbol{B}_c\boldsymbol{u}$ sampled with zero-order hold gives

$$\boldsymbol{A} = e^{\boldsymbol{A}_c T_s}, \qquad \boldsymbol{B}_d = \boldsymbol{A}_c^{-1}(\boldsymbol{A} - \boldsymbol{I})\boldsymbol{B}_c, \qquad z_k = e^{-T_s/\tau_k} \;\Leftrightarrow\; \tau_k = -\frac{T_s}{\ln z_k}. \tag{11}$$

For a thermal system the discrete poles $z_k = \mathrm{eig}(\boldsymbol{A})$ are real with $0 < z_k < 1$: $z_k \to 1$ is a slow mode, $z_k \to 0$ a fast one. Complex or negative poles indicate an over-fitted order. Compared with the RC mode the SS model shares the time constants between all channels ((5)) and includes cross-heating.

4.2 Subspace identification (N4SID/MOESP)

TherMOS uses the past-output MOESP algorithm [3, 4] (`fitN4sid`, the counterpart of MATLAB `n4sid`). With s block rows (horizon) the inputs are arranged in a block Hankel matrix

$$\boldsymbol{U} = \begin{bmatrix} \boldsymbol{u}(0) & \boldsymbol{u}(1) & \cdots & \boldsymbol{u}(J-1) \\ \boldsymbol{u}(1) & \boldsymbol{u}(2) & \cdots & \boldsymbol{u}(J) \\ \vdots & & & \vdots \\ \boldsymbol{u}(2s-1) & \cdots & & \boldsymbol{u}(2s+J-2) \end{bmatrix} = \begin{bmatrix} \boldsymbol{U}_p \\ \boldsymbol{U}_f \end{bmatrix}, \tag{12}$$

split into past (p) and future (f) halves; $\boldsymbol{Y}_p$, $\boldsymbol{Y}_f$ likewise. Repeated substitution of (10) gives the data equation

$$\boldsymbol{Y}_f = \boldsymbol{\Gamma}_s\,\boldsymbol{X}_f + \boldsymbol{H}_s\,\boldsymbol{U}_f, \qquad \boldsymbol{\Gamma}_s = \begin{bmatrix} \boldsymbol{C}_o \\ \boldsymbol{C}_o\boldsymbol{A} \\ \vdots \\ \boldsymbol{C}_o\boldsymbol{A}^{s-1} \end{bmatrix}, \qquad \boldsymbol{H}_s = \begin{bmatrix} \boldsymbol{0} & & \\ \boldsymbol{C}_o\boldsymbol{B}_d & \boldsymbol{0} & \\ \boldsymbol{C}_o\boldsymbol{A}\boldsymbol{B}_d & \boldsymbol{C}_o\boldsymbol{B}_d & \ddots \end{bmatrix} \tag{13}$$

with the extended observability matrix $\boldsymbol{\Gamma}_s$ and the Toeplitz matrix $\boldsymbol{H}_s$ of the Markov parameters. The algorithm

1. computes the LQ decomposition of $[\boldsymbol{U}_f;\ \boldsymbol{W}_p;\ \boldsymbol{Y}_f]$ with $\boldsymbol{W}_p = [\boldsymbol{U}_p;\ \boldsymbol{Y}_p]$; the block that maps $\boldsymbol{W}_p$ to $\boldsymbol{Y}_f$ after removing $\boldsymbol{U}_f$ equals $\boldsymbol{\Gamma}_s\boldsymbol{X}_f$ seen through the past data;
2. takes its singular value decomposition $\boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{V}^{\mathsf{T}}$. The number of significant singular values is the **model order**: the largest gap on a log scale is chosen when a list of orders is given;
3. sets $\boldsymbol{\Gamma}_s = \boldsymbol{U}_1\boldsymbol{\Sigma}_1^{1/2}$ and reads $\boldsymbol{C}_o$ from its first block row and $\boldsymbol{A}$ from the shift structure $\underline{\boldsymbol{\Gamma}}_s\boldsymbol{A} = \overline{\boldsymbol{\Gamma}}_s$ (all block rows but the last resp. the first) by least squares;
4. finds $\boldsymbol{B}_d$ (and the initial states $\boldsymbol{x}_0$ of the profiles) by linear least squares on the simulated output.

Several profiles are stacked column-wise, so the model is fitted to all experiments at once. If no horizon is given the horizon with the smallest simulation error is chosen from a set of candidates.

4.3 Output-error refinement

The subspace estimate minimises an algebraic criterion. Option `dis = 2` refines it with the prediction-error method in output-error form [5] (`fitPem`, MATLAB `ssest` without a noise model):

$$\min_{\boldsymbol{A},\boldsymbol{B}_d,\boldsymbol{C}_o,\boldsymbol{x}_0} \sum_e \sum_n \left\| \boldsymbol{y}_e(n) - \hat{\boldsymbol{y}}_e(n) \right\|^2, \tag{14}$$

where $\hat{\boldsymbol{y}}_e$ is the free simulation of experiment e. It has $n_x^2 + n_x m + p n_x$ parameters. With `sta = 2` unstable poles are mirrored into the unit circle ($z \to 1/z^*$) and $\boldsymbol{B}_d$, $\boldsymbol{x}_0$ are refitted.

Choosing the order

n_x is about the number of time constants that the data excite and that the sampling resolves. Too small an order merges time constants (the fast rise or the slow tail is missed); too large an order fits noise and produces complex or unstable poles. Start with the number of outputs +1 and check the singular value plot of the GUI (*Theory* view of the SS mode): a clear gap after σ_n supports order n.

Configuration in TherMOS

`Mdl > ss > K`: order n_x. `Mdl > ss > dis`: 1 subspace only, 2 subspace + output-error fit. `Mdl > ss > init`: 1 zero initial state, 2 estimated. `Mdl > ss > sta`: 2 enforces stability. `Mdl > ss > sol`: 1 free simulation from the initial state, 2 as MATLAB `compare`.

5 Coupled heat sources (SF mode)

Power modules carry several chips on one substrate; each chip heats its neighbours. The SF mode builds a nodal network of M coupled sources with K stages each:

$$\boldsymbol{C}\,\dot{\boldsymbol{\theta}} = \boldsymbol{E}\,\boldsymbol{P}(t) - \boldsymbol{G}\,\boldsymbol{\theta}, \qquad \boldsymbol{\theta}_j = [\boldsymbol{\theta}]_{\text{junctions}}, \qquad R_{\text{th}} = \left[\boldsymbol{G}^{-1}\right]_{jj}. \tag{15}$$

$\boldsymbol{C}$ is diagonal (one capacitance per node), $\boldsymbol{E}$ puts the loss of chip i into its junction node, and $\boldsymbol{G}$ is assembled from the ladder and coupling resistances:

$$\boldsymbol{G} = \sum_{i,k} \frac{1}{R'_{ik}} (\boldsymbol{e}_{ik} - \boldsymbol{e}_{i,k+1})(\boldsymbol{e}_{ik} - \boldsymbol{e}_{i,k+1})^{\mathsf{T}} + \sum_{i<j,\,k} \frac{1}{R_{ij,k}} (\boldsymbol{e}_{ik} - \boldsymbol{e}_{jk})(\boldsymbol{e}_{ik} - \boldsymbol{e}_{jk})^{\mathsf{T}}, \tag{16}$$

where $\boldsymbol{e}_{ik}$ is the unit vector of stage k of chip i and the last stage connects to T_{ref} ($\boldsymbol{e}_{i,K+1} = \boldsymbol{0}$).

Table 2: Symbols of the coupled network.

Symbol	Unit	Meaning
M, K	–	number of heat sources (chips); stages per chip (`Mdl > rc > K`)
R'_{ik}, C'_{ik}	K/W, J/K	resistance and capacitance of stage k of chip i (stage 1 = junction)
$R_{ij,k}$	K/W	coupling resistance between stage k of chips i and j (large = weak coupling)
$R_{\mathrm{th}} \in \mathbb{R}^{M\times M}$	K/W	steady-state resistance matrix; column j: rise of all junctions for 1 W in chip j
b	–	bound factor of the fit (`Mdl > sf > bnd`)

5.1 Method 2 (default): coupled Cauer network fitted to any profile

All parameters are found by a bounded output-error fit (trust-region reflective):

$$\min_{\boldsymbol{p}} \sum_{n} \left\| \boldsymbol{T}(n) - \hat{\boldsymbol{T}}(n;\boldsymbol{p}) \right\|^2, \qquad \boldsymbol{p} = \ln\!\left[R'_{ik},\ C'_{ik},\ 1/R_{ij,k}\right], \qquad \boldsymbol{p}_0 - \ln b \le \boldsymbol{p} \le \boldsymbol{p}_0 + \ln b. \tag{17}$$

The log-parameters keep every R and C positive, so every candidate is passive and stable. The start values are $R'_{ik} = R_{\mathrm{th},i}/K$ (from a least-squares fit of the last fifth of every profile), time constants spaced logarithmically between $T_s/2$ and a third of the profile length, and couplings $1/R_{ij,k} = 0.1/\overline{R}_{\mathrm{th}}$. Long data are decimated to `Nmax` samples. The network is simulated exactly for losses held constant over a sample (`sfNet`): with $\boldsymbol{S} = \boldsymbol{C}^{1/2}$ and $\boldsymbol{S}^{-1}\boldsymbol{G}\boldsymbol{S}^{-1} = \boldsymbol{V}\boldsymbol{\Lambda}\boldsymbol{V}^{\mathsf{T}}$,

$$z_k(n+1) = e^{-\lambda_k T_s} z_k(n) + \frac{1 - e^{-\lambda_k T_s}}{\lambda_k} b_k(n), \qquad \boldsymbol{b} = \boldsymbol{V}^{\mathsf{T}}\boldsymbol{S}^{-1}\boldsymbol{E}\boldsymbol{P}, \qquad \boldsymbol{\theta} = \boldsymbol{S}^{-1}\boldsymbol{V}\boldsymbol{z}. \tag{18}$$

5.2 Method 1 (MATLAB): nodal Foster model

For every chip k a Foster network is fitted to the self-heating curve of an experiment in which only chip k is heated; the steady-state matrix $R_{\mathrm{th},ik} = \Delta T_i(\infty)/P_k$ is symmetrised and inverted ($\boldsymbol{G} = R_{\mathrm{th}}^{-1}$). It needs one single-source experiment per chip. Its ODE time constants differ from the fitted Foster pairs, so its dynamics are approximate. With fewer experiments than chips TherMOS falls back to method 2.

Identifiability

Superposition gives the junction rise $\theta_i = \sum_j R_{\mathrm{th},ij} P_j$ in steady state. If two chips always carry the same loss (diagonal switches of a full bridge), the data only show the sum of their columns; the fitted network predicts this operation well, but the individual entries of R_{th} are not unique. Add single-chip steps to the training data to separate self- and cross-heating. Couplings that end at the lower bound are negligible.

Configuration in TherMOS

Setup `selSF`: 1 nodal Foster, 2 coupled Cauer network. `Mdl > rc > K`: stages per chip. `Mdl > sf > bnd`: bound factor b. `Mdl > sf > Nmax`: maximum samples of the fit.

6 Proper orthogonal decomposition and Galerkin projection (PO mode)

The lumped models reproduce a few temperatures. The PO mode reproduces the *whole field*, e.g. to find the hottest point of a board whose position changes with the load [9].

6.1 Proper orthogonal decomposition (POD)

The temperature field is approximated by K spatial modes $\phi_i(\boldsymbol{x})$ with time coefficients $a_i(t)$:

$$T(\boldsymbol{x},t) \approx T_0(\boldsymbol{x}) + \sum_{i=1}^{K} \phi_i(\boldsymbol{x})\, a_i(t), \qquad \boldsymbol{T}(t) \approx \boldsymbol{T}_0 + \boldsymbol{\Phi}\,\boldsymbol{a}(t), \quad \boldsymbol{\Phi} \in \mathbb{R}^{N\times K}. \tag{19}$$

POD chooses the basis that represents the N_t training snapshots $\boldsymbol{\theta}(t_1),\dots,\boldsymbol{\theta}(t_{N_t})$ (columns of $\boldsymbol{\Theta} \in \mathbb{R}^{N\times N_t}$, mean-centred in the PO mode) with the smallest mean square error [2]:

$$\min_{\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\Phi}=\boldsymbol{I}} \sum_t \left\|\boldsymbol{\theta}(t) - \boldsymbol{\Phi}\boldsymbol{\Phi}^\mathsf{T}\boldsymbol{\theta}(t)\right\|^2 = \sum_{i>K} \sigma_i^2, \qquad \boldsymbol{\Theta} = \boldsymbol{U}\boldsymbol{\Sigma}\boldsymbol{W}^\mathsf{T}, \quad \boldsymbol{\Phi} = \boldsymbol{U}_{:,1..K}. \tag{20}$$

The minimum is reached by the first K left singular vectors (Eckart–Young theorem); the residual is the sum of the discarded squared singular values σ_i. The *captured energy*

$$E_K = \frac{\sum_{i\le K} \sigma_i^2}{\sum_i \sigma_i^2} \tag{21}$$

selects K (`Mdl › pod › sel = 2`: smallest K with $E_K \ge$ `Emax`). Mode 1 is the mean heating pattern, the next modes describe how the sources heat differently and how the heat spreads.

POD versus modal truncation

The eigenvectors $\boldsymbol{v}_k$ of (4) are also a reduction basis (*modal truncation*: keep the slowest K). They depend only on the system. POD modes depend on the training load: they are optimal for loads like the training load but can miss a source that was never active in training. Train the POD on profiles in which every source is active with different amplitudes (random levels are better than one synchronous step). The GUI page *Theory & Tools › MOR & POD* compares both bases on a 1D rod.

6.2 Galerkin projection

Inserting (19) into the heat equation and requiring the residual to be orthogonal to every mode (weak form, test functions ϕ_i) gives K ordinary differential equations

$$\boldsymbol{C}_r\,\dot{\boldsymbol{a}} = -\boldsymbol{G}_r\,\boldsymbol{a} + \boldsymbol{\alpha} \circ \boldsymbol{q}(t), \tag{22}$$

$$C_{r,ij} = \int_\Omega \rho c\,\phi_i\phi_j\,\mathrm{d}\Omega, \quad G_{r,ij} = \int_\Omega (k\,\partial_x\phi_i\partial_x\phi_j + k_y\,\partial_y\phi_i\partial_y\phi_j + h_v\,\phi_i\phi_j)\mathrm{d}\Omega, \quad q_i = \int_\Omega Q\,\phi_i\,\mathrm{d}\Omega. \tag{23}$$

$\boldsymbol{\alpha}$ is a load correction ($\boldsymbol{\alpha} = \boldsymbol{1}$ for the pure projection). For a discretised model the same projection reads $\boldsymbol{C}_r = \boldsymbol{\Phi}^\mathsf{T}\boldsymbol{C}\boldsymbol{\Phi}$, $\boldsymbol{G}_r = \boldsymbol{\Phi}^\mathsf{T}\boldsymbol{G}\boldsymbol{\Phi}$, $\boldsymbol{q} = \boldsymbol{\Phi}^\mathsf{T}\boldsymbol{B}\boldsymbol{P}$; it keeps symmetry and positive definiteness, so the projected model is stable and passive. In the PO mode the modes are given at scattered FE nodes; they are interpolated to a regular grid and the integrals are evaluated with Gauss–Legendre quadrature ($\int f\,\mathrm{d}\Omega \approx \sum_q W_q f(\boldsymbol{x}_q)$, `calcPodQuad`, `calcPodProj`). The ODE is solved exactly with the matrix exponential for losses linear between samples, starting from $\boldsymbol{a}(0) = \boldsymbol{0}$ at the measured initial field $\boldsymbol{T}_0$.

Assumptions. Mean-centred modes vanish on fixed-temperature boundaries, so no boundary term appears. Convective boundaries must be part of $\boldsymbol{G}_r$ (h_v for boards): without the heat sink the projected model has no path for the heat to leave, and the temperature rises without bound (Table 4).

6.3 Calibration protocols

FE data often do not contain every physical detail (contact resistances, the real cooling), and the interpolation and quadrature add errors. Protocols 4–6 therefore correct the projected matrices with the training data. With the modal coordinates $\boldsymbol{a}(t) = \boldsymbol{\Phi}^{\mathsf{T}}(\boldsymbol{T}(t) - \boldsymbol{T}_0)$, their central-difference derivatives $\dot{\boldsymbol{a}}$ and the projected loads $\boldsymbol{q}$, all problems are linear least squares (equation error, `optPodMat`):

Table 3: POD protocols (`Mdl > pod > protocol`).

No.	Reduced matrices	Use
1	unweighted sum over the nodes	MATLAB behaviour only (`legacy = 1`)
2	Gauss–Legendre projection (pure Galerkin)	physics only; extrapolates safely if the data contain all heat paths
3	as 2 (legacy: relative element size)	physics only
4	2 + steady-state load correction $\min_{\boldsymbol{\alpha}} \lVert \boldsymbol{G}_r\boldsymbol{a} - \boldsymbol{\alpha} \circ \boldsymbol{q} \rVert^2$	physics + one calibration vector
5	$\min_{\boldsymbol{C}_r,\boldsymbol{G}_r} \lVert \boldsymbol{q} - \boldsymbol{C}_r\dot{\boldsymbol{a}} - \boldsymbol{G}_r\boldsymbol{a} \rVert^2$, then $\boldsymbol{\alpha}$	data-calibrated (default), most accurate on FE or measured data
6	as $5 + \lambda\big(\lVert \boldsymbol{C}_r - \boldsymbol{C}_{r0} \rVert^2 + \lVert \boldsymbol{G}_r - \boldsymbol{G}_{r0} \rVert^2\big)$, $\lambda = 10^{-3}$	regularised towards the projection; robust with little data

Protocol 5 is underdetermined for modes with almost no energy and for pure step responses (the regressors $\dot{\boldsymbol{a}}$ and $\boldsymbol{a}$ then span only the exponentials of the response and cannot separate $\boldsymbol{C}_r$ from $\boldsymbol{G}_r$). TherMOS solves it per mode with a truncated SVD (singular values below 10^{-3} of the largest are dropped) as the minimum-norm correction of the projected matrices, and keeps the projected matrices if the calibrated model is singular or unstable.

Worked example 2: POD-Galerkin of a DBC board

Board designer model of a 40 mm × 30 mm DBC (Al_2O_3 on a 3 mm Cu baseplate, $h = 1000\,\mathrm{W/(m^2\,K)}$, $T_a = 25\,^\circ\mathrm{C}$) with an IGBT (40 W) and a diode (15 W) on a 1 mm grid, $N = 1271$ nodes. Training: 600 s of random loss levels; test: pulse trains with periods of 120 s and 200 s (maximum rise 50.5 K). The snapshots are not centred here ($\theta = T - T_a$). Table 4 lists the result; Fig. 4 shows the singular values and the error.

Table 4: Worked example 2: maximum absolute error on the test profile over the number of modes K. "Best possible" is the projection of the true test field onto the K modes; "without h_v" omits the cooling from $\boldsymbol{G}_r$.

K	$1 - E_K$	Galerkin (K)	best possible (K)	without h_v (K)
1	7.6×10^{-4}	4.40	4.61	494
2	3.7×10^{-5}	2.13	2.40	538
3	1×10^{-6}	0.87	0.71	987
5	$< 10^{-6}$	0.041	0.027	985
8	$< 10^{-6}$	0.0042	0.0013	985

Interpretation. The energy criterion alone is misleading here: 99.92 % energy with one mode still leaves a 4.4 K error at the hot spot, because the energy is dominated by the large mean heating pattern and the hot spot is a small region. Choose K from the error at the temperatures that

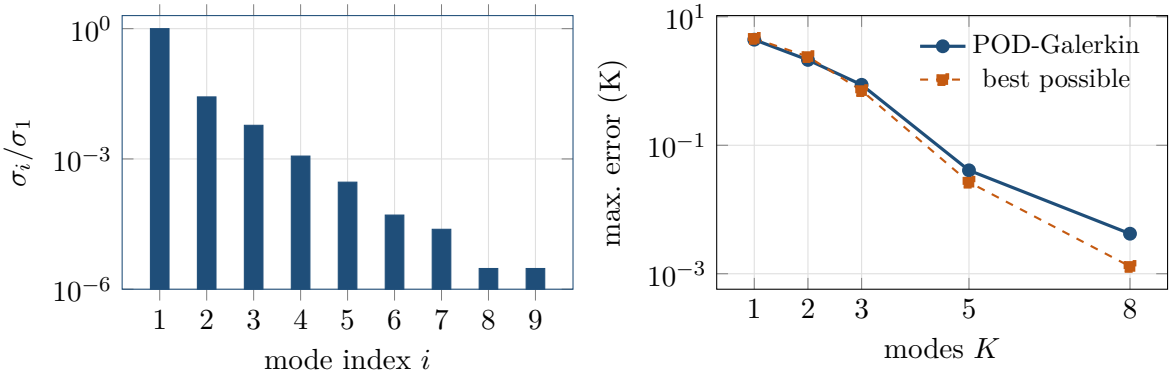


Figure 4: Worked example 2. Left: the singular values fall by more than four decades within the first six modes – two sources and one cooling path need only a few spatial patterns. Right: the Galerkin error falls from 4.4 K with one mode to 0.04 K (0.08 % of the rise) with five modes and stays within a factor of about 3 of the best error the basis allows. The full model has 1271 states, the reduced model 5: a reduction by a factor of 250 at an error far below the accuracy of the material data.

matter (junctions), not from E_K alone; $E_K \geq 1 - 10^{-6}$ was needed here for an error below 0.1 K. The column "without h_v" shows why the projection must contain every heat sink.

Configuration in TherMOS

`Mdl > pod > K`: modes K (`sel = 1`). `Mdl > pod > sel`: 1 fixed K, 2 energy criterion `Mdl > gen > Emax` with $K \leq$ `Kmax`. `Mdl > pod > protocol`: Table 3. GUI: *Theory & Tools › Numerics* explains the quadrature weights and the calibration.

7 POD state space (PS mode)

The PS mode keeps the POD modes but *identifies* the dynamics of the modal coordinates instead of projecting the heat equation:

$$\boldsymbol{a}(n) = \boldsymbol{\Phi}^{\mathsf{T}}(\boldsymbol{T}(n) - \boldsymbol{T}_0), \qquad q_i(n) = \int_\Omega Q(n)\,\phi_i \,\mathrm{d}\Omega, \tag{24}$$

$$\boldsymbol{x}(n+1) = \boldsymbol{A}\,\boldsymbol{x}(n) + \boldsymbol{B}_d\,\boldsymbol{q}(n), \qquad \boldsymbol{a}(n) = \boldsymbol{C}_o\,\boldsymbol{x}(n), \qquad \boldsymbol{T}(n) = \boldsymbol{T}_0 + \boldsymbol{\Phi}\,\boldsymbol{C}_o\,\boldsymbol{x}(n), \tag{25}$$

with K modes and K_s states, identified as in Section 4 (subspace + output-error fit; every profile is preceded by $2K_s$ zero samples for a zero initial state). The implementation is dimension independent (2D and 3D).

PO or PS?

PS needs no material data and no boundary conditions – only snapshots and losses – and absorbs model errors of the data (unknown cooling, FE mesh effects). It only knows the dynamics that the training loads excited and has no physical structure that guarantees stability or passivity. PO with protocol 2 extrapolates most safely when the physics in the data is complete; PO with protocol 5 is a compromise. The slowest time constants of both should agree: they are set by the heat capacity and the cooling.

Configuration in TherMOS

`Mdl > pod > K`, `sel`, `Emax` as in the PO mode; `Mdl > ss > K`: states K_s; `Mdl > ss > dis/init/sta` as in the SS mode. Start with $K_s \approx K$ to $2K$.

8 Reduced models of the board designer

The board designer (GUI, *Theory & Tools*) builds a layer-resolved FV model of a board with chips, thermal vias and layers below (TIM, heat sink) and reduces it in two ways.

8.1 POD-Galerkin on the FV grid

The snapshots of a training run give $\boldsymbol{\Phi}$; the FV matrices are projected directly (`romBoard`): $\boldsymbol{C}_r = \boldsymbol{\Phi}^\mathsf{T}\mathrm{diag}(\boldsymbol{C})\boldsymbol{\Phi}$, $\boldsymbol{G}_r = \boldsymbol{\Phi}^\mathsf{T}\boldsymbol{G}\boldsymbol{\Phi}$, $\boldsymbol{B}_r = \boldsymbol{\Phi}^\mathsf{T}\boldsymbol{B}$. The reduced model is integrated with the same implicit Euler scheme as the full model, so the difference is the projection error only (worked example 2).

8.2 Coupled Cauer network from step responses

The *thermal characterisation* computes the response of every junction to a 1 W step in every chip, $Z_{\mathrm{th},ij}(t)$, on logarithmically spaced times up to the slowest time constant, and the steady-state matrix R_th. A coupled Cauer network (Section 5) with K stages per chip is fitted to these step responses; its matrices $\boldsymbol{G}_\mathrm{th}$, $\boldsymbol{C}_\mathrm{th}$, $\boldsymbol{E}$ can be exported and used e.g. as the physics of a PINN (Section 10). The junction temperature is either the die average (linear in the losses, superposition is exact, consistent with data sheets) or the hottest cell of the die (not linear: the maximum of a sum is not the sum of the maxima, superposition over-estimates it when several chips heat).

8.3 Electro-thermal coupling

Losses rise with temperature. Both reduced models can include the linear coupling $P_{i,\mathrm{eff}} = P_i \max\bigl(1+\alpha_i(T_{j,i}-T_\mathrm{ref}),0\bigr)$. The steady state solves $(\boldsymbol{G}-\boldsymbol{U}\boldsymbol{W}^\mathsf{T})\boldsymbol{\theta} = \boldsymbol{B}\boldsymbol{P}(1+\boldsymbol{\alpha}(T_a-T_\mathrm{ref}))$ with $\boldsymbol{U} = \boldsymbol{B}\,\mathrm{diag}(\boldsymbol{\alpha}\boldsymbol{P})$ (Woodbury identity); a loop gain ≥ 1 means thermal runaway.

9 Data-driven models (ML and DL modes)

When no geometry is available – e.g. an electric motor with only a few temperature sensors and operating data (currents, voltages, speed, coolant temperature, as in the motor data set of the ML/DL tutorials [8]) – the input–output map is learned from profiles.

9.1 Static regression with smoothed features (ML mode)

The ML mode fits a static regression $\hat{T}(n) = f(\boldsymbol{X}(n))$ from the features of one sample to the temperature of the first node: (1) linear regression, (2) regression tree, (3) support vector regression, (4) Gaussian process regression, (5) ensemble (boosting or bagging), (6) multilayer perceptron. A static map has no memory; the thermal memory enters through exponentially weighted moving averages (EWMA) and standard deviations (EWMS) of every feature (`calcFeat`):

$$\mu(n) = \alpha\,x(n)+(1-\alpha)\,\mu(n-1), \qquad \sigma^2(n) = (1-\alpha)\bigl(\sigma^2(n-1)+\alpha\,(x(n)-\mu(n-1))^2\bigr), \qquad \alpha = \frac{2}{s+1}. \tag{26}$$

The EWMA is a first-order low-pass filter – the discrete counterpart of one RC pair – with time constant $\tau = -T_s/\ln(1-\alpha) \approx sT_s/2$: a span of $s = 100$ samples is a 50 s RC pair at $T_s = 1\,\mathrm{s}$. A set of spans therefore plays the role of the Foster time constants, and the regression the role of the weights R_k – generalised to nonlinear dependencies (e.g. losses $\propto I^2$). Choose spans that cover the thermal time constants of the system.

9.2 Sequence networks (DL mode)

The DL mode maps a window of W samples of F features ($W \times F$, z-score normalised) to the temperatures of all M nodes with a convolutional (CNN), recurrent (LSTM, GRU) or attention (transformer) network or a user-defined layer list. *seq2point* (`Mdl > dl > seq` $= s > 0$) predicts sample s of the window ($s = W$: causal, only past inputs); *seq2seq* ($s = 0$) predicts every sample of the window. The window must cover the dominant time constant ($WT_s \gtrsim \tau_{\max}$), otherwise the network cannot know the thermal history. Training uses Adam with a learning-rate schedule, gradient clipping and early stopping on the validation loss; the network with the lowest validation loss is kept.

Strengths and limits

Data-driven models capture nonlinearities (temperature-dependent losses, speed-dependent cooling) that linear reduced models cannot. They need much data covering the whole operating range, they give no physical parameters, and they can fail silently outside the training range. Always test on profiles that were not used for training or validation.

Configuration in TherMOS

ML: setup `selML` (1–6), `featRoll`, `EWMA`, `EWMS`; `Mdl > gen > opt = 1` random search of the hyperparameters (3-fold cross validation). DL: setup `selDL` or `archDL`; `Mdl > dl > W, seq, stride, epoch, batch, initLr, lrAdapt, lrDropPr, lrDropFa, gradTh, valFreq, valPat, loss`; `Mdl > gen > dev = 1` GPU.

10 Physics-informed neural network (PI mode)

The PINN [7] combines a sequence network with a lumped thermal network. The network predicts the states $\boldsymbol{x}$ of the physics model

$$\boldsymbol{C}\,\dot{\boldsymbol{x}} = \boldsymbol{E}\,\boldsymbol{P}(t) - \boldsymbol{G}\,\boldsymbol{x}, \qquad \hat{T}_i = T_{\mathrm{ref},i} + (\boldsymbol{O}\boldsymbol{x})_i, \tag{27}$$

where the physics is (0) none, (1) Foster chains between sources and outputs (one state per term, $\tau_k \dot{\theta}_k = R_k P_j - \theta_k$) or (2) a Cauer/nodal network with hidden nodes (e.g. from the SF mode or the board designer). The losses P_j are equations of the input features (e.g. $3R_sI^2$ for copper losses). Training minimises three dimensionless parts (ΔT: temperature scale of the training data):

$$L = \alpha \underbrace{\operatorname{mean}\left(\frac{\hat{T}-T}{\Delta T}\right)^2}_{L_{\mathrm{data}}} + \beta \underbrace{\operatorname{mean}\left(\frac{\boldsymbol{C}\dot{\boldsymbol{x}}+\boldsymbol{G}\boldsymbol{x}-\boldsymbol{E}\boldsymbol{P}}{\operatorname{diag}(\boldsymbol{G})\,\Delta T}\right)^2}_{L_{\mathrm{phys}}} + \gamma \underbrace{\frac{\sum w \left(\frac{\hat{T}-T(0)}{\Delta T}\right)^2}{\sum w}}_{L_{\mathrm{init}}}, \quad w = e^{-t/\tau_{\min}}. \tag{28}$$

L_{data} is evaluated where temperatures are measured, L_{phys} for every state and also on *collocation profiles* (inputs without measured temperatures, `Mdl > pinn > col`), L_{init} keeps the prediction at the measured start temperature. The first `warm` samples of every window are excluded (the recurrent state settles).

When the physics helps

The physics loss acts as a regulariser: with little labelled data and an accurate physics model it constrains the network where no temperatures are measured (example in the GUI theory view: one labelled profile, test RMSE 12 K → 4 K with six unlabelled profiles). If the physics is simplified, a large β makes the network follow the wrong physics; $\beta = 0.01$–1 is the useful range.

Configuration in TherMOS

`Mdl > pinn > phys` (0/1/2), `alpha`, `beta`, `gamma`, `warm`, `pred`, `x0`, `col`, `TrefSrc`, `Tref`, `node`; setup `selPI` or `archPI`, `physPI` (GUI view *Physics*).

11 Comparison and selection guide

Table 5: Properties of the reduced model families. "Extrapolation" rates how safely the model predicts loads that differ from the training loads (+++: physics-based, ++: linear identified, +: learned).

Mode	Needs	Returns	Parameters	Coupl.	Extrap.	Typical use
RC	1 loss, 1 temperature	1 temperature	$2K$	no	++	data-sheet Z_{th}, single chip, lifetime
SS	all losses and temperatures	p temperatures	$n_x^2 + n_x(m+p)$	yes	++	MIMO, observers, control
SF	profiles of M chips	M junctions	$\approx M(2K) + KM(M-1)/2$	yes	++	power modules, inverter legs
PO	FE snapshots + material data	full field	$\approx 2K^2 + K$	yes	+++ (prot. 2)	hot spots, sensor placement
PS	FE snapshots + losses	full field	$K_s^2 + 2K_sK$ + modes	yes	++	field without material data
ML	operating data	1 temperature	model-dependent	–	+	motors, quick static estimates
DL	much operating data	M temperatures	10^4–10^5	–	+	nonlinear systems, rich data
PI	data + lumped physics	M temperatures	10^4–10^5	yes	+ to ++	little data, known structure

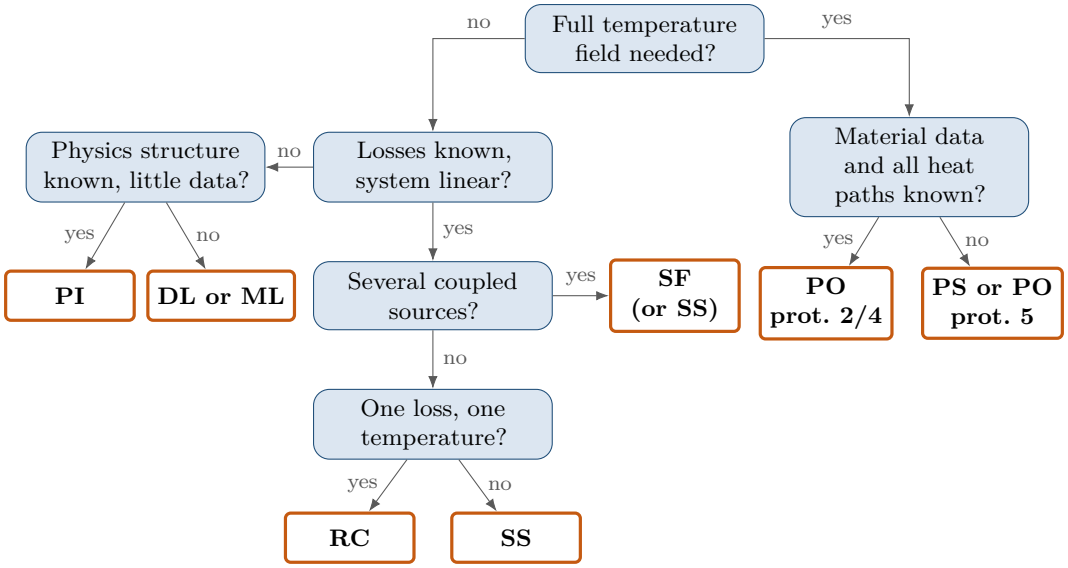


Figure 5: Selection guide. "Linear" means that the losses are known inputs and the material properties hardly depend on temperature; temperature-dependent losses can still be handled by the electro-thermal coupling of the board designer or by the loss equations of the PINN.

Practical recommendations.

- Start simple: an RC or SS model on the same data gives a baseline for every more complex model.

- Sample at least 3–5 times faster than the fastest time constant you need; record at least 3–5 times the slowest one.
- Excite every source on its own at least once (single-source steps or random levels with independent amplitudes): otherwise cross-heating cannot be separated from self-heating (Section 5).
- Choose the order from the validation error at the temperatures that matter, not from the training error or the POD energy alone.
- Test on a profile that differs from the training profiles (different frequency content and amplitudes).

Other MOR methods (not implemented)

For completeness: *balanced truncation* keeps the states that are both well controllable and well observable (Hankel singular values) and has an a-priori error bound [1]; *Krylov / moment matching* methods (PRIMA, Arnoldi) match the low-frequency behaviour of large FE models and keep passivity [1]. Both need the system matrices of the full model; TherMOS works from data (snapshots, profiles), which is why it uses POD and system identification.

12 Error metrics

All modes report the same metrics per output channel (`calcError`) with $e = y - \hat{y}$ over N samples:

Metric	Definition	Unit	Meaning
MAE	$\frac{1}{N}\sum \lvert e\rvert$	K	mean absolute error; typical deviation
RMSE	$\sqrt{\frac{1}{N}\sum e^2}$	K	root mean square error; weights large errors more
MAX	$\max \lvert e\rvert$	K	largest deviation; relevant for temperature limits
SAE	$(\overline{\lvert y\rvert} - \overline{\lvert \hat{y}\rvert})/\overline{\lvert y\rvert}$	p.u.	relative error of the mean level (sign: bias)
NAE	$\sum \lvert e\rvert / \sum \lvert y\rvert$	p.u.	absolute error relative to the signal
FIT	$100\,(1 - \lVert e\rVert / \lVert y - \bar{y}\rVert)$	%	fit percentage as MATLAB `compare`; 100 = perfect, 0 = as good as the mean
NRMSE	$100\,\mathrm{RMSE}/(\max y - \min y)$	%	RMSE relative to the temperature swing

For temperatures in °C the relative metrics SAE and NAE depend on the offset of the scale; compare models with MAE, RMSE, MAX and NRMSE.

References

[1] A. C. Antoulas, *Approximation of Large-Scale Dynamical Systems*. SIAM, 2005.

[2] P. Holmes, J. L. Lumley, G. Berkooz, C. W. Rowley, *Turbulence, Coherent Structures, Dynamical Systems and Symmetry*, 2nd ed. Cambridge University Press, 2012.

[3] M. Verhaegen, V. Verdult, *Filtering and System Identification: A Least Squares Approach*. Cambridge University Press, 2007.

[4] P. Van Overschee, B. De Moor, *Subspace Identification for Linear Systems*. Kluwer, 1996.

[5] L. Ljung, *System Identification: Theory for the User*, 2nd ed. Prentice Hall, 1999.

[6] JEDEC, *JESD51-14: Transient Dual Interface Test Method for the Measurement of the Thermal Resistance Junction-to-Case of Semiconductor Devices with Heat Flow Through a Single Path*, 2010.

[7] M. Raissi, P. Perdikaris, G. E. Karniadakis, “Physics-informed neural networks,” *J. Comput. Phys.*, vol. 378, pp. 686–707, 2019.

[8] W. Kirchgässner, O. Wallscheid, J. Böcker, “Electric Motor Temperature” data set, Kaggle, CC BY-SA 4.0.

[9] P. A. Schirmer, “TherMOS: A Robust Low-Resolution 2D Spatial Thermal Model Order Reduction Approach” (setup files in `setup/journal`).